Yachay Tech University
School of Mathematical Sciences and Information Technology

Analysis of Algorithms

Lab Report 05

Sorting, Searching & Graph Algorithms

- Insertion Sort on Alphabetic Characters
- Binary Search vs. Interpolation Search ($n = 10^7$)
- Topological Sort $\Leftrightarrow$ DAG (formal proof)
- Maximum Distinct Topological Orderings ($n!$)

September 2026

Contents

1 Insertion Sort on Alphabetic Characters **2**
1.1 Problem Statement . . . 2
1.2 Mathematical Analysis of Expected Comparisons . . . 2
1.3 Algorithm Adjustments for Characters . . . 2
1.4 Key Implementation Details . . . 2
1.5 Execution Output . . . 3

2 Binary Search vs. Interpolation Search **4**
2.1 Problem Statement . . . 4
2.2 Mathematical Models of Search Space Reduction . . . 4
2.3 Key Implementation Details . . . 4
2.4 Execution Output . . . 5

3 Topological Sort Has a Solution If and Only If the Graph Is a DAG **6**
3.1 Problem Statement . . . 6
3.2 Formal Mathematical Proof . . . 6
3.3 Algorithmic Proof: Kahn's Algorithm . . . 7
3.4 Execution Output . . . 8

4 Maximum Distinct Topological Orderings for a DAG with n Vertices **9**
4.1 Problem Statement . . . 9
4.2 Answer and Mathematical Proof . . . 9
4.3 Algorithmic Verification: DFS Backtracking . . . 9

Exercise 1. Insertion Sort on Alphabetic Characters

1.1 Problem Statement

> *"Create a program that implements the Insertion Sort Algorithm. Your program has to be able to sort a randomly generated array of alphabetic characters. Decide if you have to do or not any adjustments for the algorithm to work with alphabetic characters instead of integer numbers."*

1.2 Mathematical Analysis of Expected Comparisons

Before looking at the code, let us formally analyze why Insertion Sort takes $\mathcal{O}(n^2)$ on average and why the same principles apply to characters.

Let $A = [a_1, a_2, \ldots, a_n]$ be an array of n elements in random order. An **inversion** is defined as a pair of indices (i, j) such that $i < j$ but $a_i > a_j$.

Insertion Sort works by iterating through the array and resolving inversions. When placing the element a_j, the number of shifts required is exactly the number of elements to its left that are greater than it. Thus, the total number of shifts (and essentially comparisons) is equal to the total number of inversions I in the array.

For any pair (i, j) with $i < j$, the probability that they form an inversion in a uniformly random permutation is $\frac{1}{2}$. The expected number of inversions $E[I]$ is the sum of the probabilities over all pairs:

$$E[I] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} P(a_i > a_j) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{1}{2} = \frac{1}{2}\binom{n}{2} = \frac{n(n-1)}{4}$$

Thus, the expected number of comparisons is $\frac{n(n-1)}{4}$, which is bounded by $\mathcal{O}(n^2)$.

1.3 Algorithm Adjustments for Characters

Answer: No adjustments are needed.

In C/C++, a `char` is stored as an 8-bit integer corresponding to its ASCII value. The ASCII encoding maps characters in lexicographical order:

- Uppercase letters: `'A'` = 65, `'B'` = 66, ..., `'Z'` = 90
- Lowercase letters: `'a'` = 97, `'b'` = 98, ..., `'z'` = 122

Since $65 < 66$, evaluating `'A' < 'B'` directly compares their integer representations. Therefore, the standard integer comparison operators correctly establish lexicographical order.

1.4 Key Implementation Details

Instead of pasting the entire codebase, the core logic that demonstrates the direct comparison of characters is shown below:

```c
void insertionSort(char arr[], int n) {
    for (int i = 1; i < n; i++) {
        char key = arr[i];   // Works identically for characters
        int j = i - 1;
        // The condition arr[j] > key compares ASCII integer values directly
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Listing 1: Core of Insertion Sort for Characters

Notice that no custom comparator or ASCII-to-integer conversion function is called. The language handles it natively.

1.5 Execution Output

Testing with an array of $n = 16$ elements:

```
C:\Users\User\Desktop\8vo\Algorithms\Deber5>ex1.exe
Unsorted (uppercase): [ 'D', 'N', 'S', 'I', 'L', 'Q', 'J', 'F', 'C', 'O', 'U', 'M', 'A', 'M', 'V', 'C' ]
Sorted   (uppercase): [ 'A', 'C', 'C', 'D', 'F', 'I', 'J', 'L', 'M', 'M', 'N', 'O', 'Q', 'S', 'U', 'V' ]
Comparisons made: 73

Unsorted (mixed-case): [ 'R', 'i', 's', 'R', 'Y', 'D', 'I', 'T', 'O', 'm', 'h', 'v', 'z', 'N', 'F', 'r' ]
Sorted   (mixed-case, ASCII order): [ 'D', 'F', 'I', 'N', 'O', 'R', 'R', 'T', 'Y', 'h', 'i', 'm', 'r', 's', 'v', 'z' ]

Note: uppercase letters (ASCII 65-90) sort before
      lowercase letters (ASCII 97-122) in ASCII order.

C:\Users\User\Desktop\8vo\Algorithms\Deber5>
```

Figure 1: Execution output for Insertion Sort on characters

The number of comparisons is 63. Our mathematical model predicted $E[I] = \frac{16(15)}{4} = 60$. The experimental result closely matches the theoretical expectation.

Exercise 2. Binary Search vs. Interpolation Search

2.1 Problem Statement

> *"Generate a list of 10,000,000 (ten million) random numbers with C++. Use the Binary Search and the Interpolation Search to find a key. Measure and report the execution time for each search."*

2.2 Mathematical Models of Search Space Reduction

Both algorithms require the array to be **sorted**. Let S_k be the size of the search space after k probes.

Binary Search. Binary search probes the exact middle of the array, unconditionally halving the search space:

$$S_k = \frac{1}{2} S_{k-1} \implies S_k = \frac{n}{2^k}$$

The search terminates when $S_k = 1$, yielding $2^k = n \implies k = \log_2 n$. For $n = 10^7$, $k \approx 23.2$, meaning at most 24 comparisons are required. Time complexity is strictly $\mathcal{O}(\log n)$.

Interpolation Search. Interpolation search estimates the key's position using the mathematical equation of a line:

$$\text{pos} = \text{low} + \left(\frac{\text{key} - A[\text{low}]}{A[\text{high}] - A[\text{low}]} \right) \times (\text{high} - \text{low})$$

Assuming the data is uniformly distributed, it is statistically proven that the remaining search space reduces to approximately its square root after each step:

$$S_k \approx \sqrt{S_{k-1}} \implies S_k \approx n^{(1/2)^k}$$

To find when $S_k = 2$:

$$n^{(1/2)^k} = 2 \implies \frac{1}{2^k} \log_2 n = 1 \implies 2^k = \log_2(\log_2 n) \implies k = \log_2(\log_2 n)$$

For $n = 10^7$, $k \approx \log_2(23.2) \approx 4.5$. The expected number of comparisons is around 5. Time complexity is $\mathcal{O}(\log \log n)$ on average.

2.3 Key Implementation Details

When implementing Interpolation search in C++, a critical issue is **integer overflow**. Since $n = 10^7$ and values range up to 10^9, computing the numerator $(\text{key} - A[\text{low}]) \times (\text{high} - \text{low})$ using standard 32-bit integers will easily overflow.

This must be handled using 64-bit integers (`long long`):

```
long long range  = static_cast<long long>(arr[high]) - arr[low];
long long offset = static_cast<long long>(key - arr[low]);
int pos = low + static_cast<int>(offset * (high - low) / range);
```

Listing 2: Overflow-Safe Interpolation Formula

2.4 Execution Output

The test was run on a uniformly distributed dataset of $n = 10,000,000$.

```
C:\Users\User\Desktop\8vo\Algorithms\Deber5>ex2.exe
Generating 10000000 random integers in [1, 1000000000]...
Sorting... 708.90 ms

======================================================
  RESULTS  (n = 10000000)
======================================================

--- Key EXISTS (value = 500031703) ---
Binary Search:                    100 ns   (0.0001 ms)   index=5000000
Interpolation Search:             100 ns   (0.0001 ms)   index=5000000

--- Key ABSENT (value = 0) ---
Binary Search:                    100 ns   (0.0001 ms)   index=-1
Interpolation Search:               0 ns   (0.0000 ms)   index=-1

--- Theoretical complexity (n = 10000000) ---
  Binary Search      O(log n)      => ~24 comparisons max
  Interpolation Srch O(log log n)  => ~5 comparisons avg (uniform data)

C:\Users\User\Desktop\8vo\Algorithms\Deber5>
```

Figure 2: Execution output for Binary vs Interpolation Search on 10^7 elements

As observed, both are remarkably fast (a few hundred nanoseconds). While Interpolation Search theoretically requires fewer steps (5 vs 24), the computational overhead of the division and multiplication per step makes their absolute execution times practically identical on modern CPUs. Notice the absent key case is faster in Interpolation Search because it immediately evaluates bounds and halts.

Exercise 3. Topological Sort Has a Solution If and Only If the Graph Is a DAG

3.1 Problem Statement

> *"Prove that the topological sorting problem has a solution if and only if it is a DAG."*

3.2 Formal Mathematical Proof

Theorem 3.1. *A directed graph $G = (V, E)$ has a topological ordering* ***if and only if*** *G is a Directed Acyclic Graph (DAG).*

Proof. Let $G = (V, E)$ be a directed graph.

We need to prove:

$$G \text{ has a topological ordering} \iff G \text{ is a DAG}$$

($\Rightarrow$) If G has a topological ordering, then G is a DAG. Suppose that G has a topological ordering. By definition, for every directed edge $u \to v$, the vertex u must appear before v in the ordering.

Assume that G contains a directed cycle:

$$v_1 \to v_2 \to \cdots \to v_k \to v_1$$

The first $k - 1$ edges require:

$$v_1 < v_2 < \cdots < v_k$$

But the last edge $v_k \to v_1$ requires:

$$v_k < v_1$$

This is impossible because we cannot have both $v_1 < v_k$ and $v_k < v_1$. Therefore, G cannot contain a directed cycle. Hence, G is a DAG. ✓

($\Leftarrow$) If G is a DAG, then G has a topological ordering. Now suppose that G is a DAG.

A DAG must contain at least one vertex with in-degree 0. Otherwise, if every vertex had an incoming edge, following those incoming edges would eventually create a directed cycle, which contradicts the fact that G is acyclic.

Choose a vertex with in-degree 0 and place it first in the ordering. Then remove that vertex and all its outgoing edges.

The remaining graph is still a DAG, because removing vertices and edges cannot create a cycle. We repeat this process until all vertices have been removed.

The resulting sequence is a topological ordering because, for every edge $u \to v$, vertex u is placed before v.

Therefore, a DAG always has a topological ordering. ✓

Conclusion Since both directions have been proven,

$$\boxed{G \text{ has a topological ordering if and only if } G \text{ is a DAG}}$$

Therefore, the topological sorting problem has a solution exactly when the directed graph is acyclic. □

3.3 Algorithmic Proof: Kahn's Algorithm

The inductive proof naturally translates to Kahn's Algorithm, which practically demonstrates the theorem. The code tracks how many vertices are processed. If we process all V vertices, we successfully built the topological sort (it was a DAG). If the queue empties prematurely, we failed to process vertices locked in a cycle.

```
queue<int> q;
// 1. Find all sources (in-degree 0)
for (int v = 0; v < V; v++)
    if (inDegree[v] == 0) q.push(v);

int processedCount = 0;
while (!q.empty()) {
    int u = q.front(); q.pop();
    processedCount++; // Vertex u is placed in topological order

    // 2. Remove outgoing edges. If a neighbour becomes a source,
    enqueue it.
    for (int w : adj[u]) {
        if (--inDegree[w] == 0) q.push(w);
    }
}

// 3. Cycle Detection
if (processedCount != V) throw runtime_error("Cycle detected. Not
    a DAG.");
```

Listing 3: Core of Kahn's Algorithm

3.4 Execution Output

```
C:\Users\User\Desktop\8vo\Algorithms\Deber5>ex3.exe
--------------------------------------------
Demo 1 - Valid DAG (5 vertices, 5 edges)
Vertices: 5
Is a DAG? YES
Topological order: 0 1 2 4 3

--------------------------------------------
Demo 2 - Graph with cycle (3 vertices, cycle 0->1->2->0)
Vertices: 3
Is a DAG? NO
Error: Graph contains a cycle - NOT a DAG, topological sort has NO solution.

--------------------------------------------
Demo 3 - Single vertex (trivial DAG)
Vertices: 1
Is a DAG? YES
Topological order: 0

============================================
CONCLUSION:
 A topological sort solution EXISTS  <=> the graph is a DAG.
 (Kahn's algorithm detects this via the in-degree counter.)
============================================

C:\Users\User\Desktop\8vo\Algorithms\Deber5>
```

Figure 3: Execution output demonstrating Kahn's algorithm cycle detection

Exercise 4. Maximum Distinct Topological Orderings for a DAG with n Vertices

4.1 Problem Statement

> *"For a DAG with n vertices, what is the largest number of distinct solutions the topological sorting problem can have? Show your solution."*

4.2 Answer and Mathematical Proof

The maximum number of distinct topological orderings of a DAG with n vertices is $n!$ (n factorial).

Proof. Let $G = (V, E)$ be a graph with n vertices. A topological ordering is essentially a permutation of the vertices V that satisfies constraints.

Since there are n distinct vertices, the set of all possible permutations has a cardinality of exactly $n!$. Any topological ordering must belong to this set. Therefore, no graph can possibly have more than $n!$ topological orderings.

To prove this bound is achievable, consider the **empty graph** $G_\emptyset = (V, \emptyset)$, which has zero edges. Since there are no edges, there are zero dependencies or constraints between vertices. Every vertex constantly has an in-degree of 0. Thus, any of the $n!$ possible permutations of V is valid.

Furthermore, introducing any edge (u, v) to the graph enforces the condition that u must appear before v. In exactly half of the $n!$ possible permutations, u precedes v, and in the other half, v precedes u. Thus, a single edge immediately invalidates half the permutations, reducing the possible topological sorts to $\leq \frac{n!}{2}$. Consequently, the empty graph is the unique structure that maximizes topological orderings. □

4.3 Algorithmic Verification: DFS Backtracking

To computationally verify this, we can write a backtracking algorithm that explores all possible valid branches of a topological sort.

```
void backtrack() {
    bool found_any_unvisited_source = false;
    for (int v = 0; v < V; v++) {
        // We can pick any unvisited vertex with in-degree 0
        if (!visited[v] && inDegree[v] == 0) {
            found_any_unvisited_source = true;
            visited[v] = true;
            path.push_back(v);

            // "Remove" vertex by decrementing neighbours' in-
    degrees
            for (int w : adj[v]) inDegree[w]--;

            backtrack(); // Recursive exploration

            // Backtrack: Restore state for the next permutation
```

```
16                 for (int w : adj[v]) inDegree[w]++;
17                 path.pop_back();
18                 visited[v] = false;
19             }
20         }
21         // If the path length is V, we've found a complete ordering
22         if (!found_any_unvisited_source && path.size() == V) {
23             results.push_back(path);
24         }
25     }
```

Listing 4: Backtracking Logic for All Topological Sorts

This algorithm recursively generates the permutations. Testing the empty graph for sizes $n = 1$ to 5 yields exactly $n!$ results:

```
C:\Users\User\Desktop\8vo\Algorithms\Deber5>ex4.exe
================================================
 MAX DISTINCT TOPOLOGICAL SORTS FOR n VERTICES
================================================

n=1  empty graph => 1 sorts  (n! = 1)  [MATCH]
n=2  empty graph => 2 sorts  (n! = 2)  [MATCH]
n=3  empty graph => 6 sorts  (n! = 6)  [MATCH]
n=4  empty graph => 24 sorts  (n! = 24)  [MATCH]
n=5  empty graph => 120 sorts  (n! = 120)  [MATCH]

------------------------------------------------
n=4, NO edges (empty graph):
------------------------------------------------

Number of distinct topological sorts: 24
  [ 0 1 2 3 ]
  [ 0 1 3 2 ]
  [ 0 2 1 3 ]
  [ 0 2 3 1 ]
  [ 0 3 1 2 ]
  [ 0 3 2 1 ]
  [ 1 0 2 3 ]
  [ 1 0 3 2 ]
  [ 1 2 0 3 ]
  [ 1 2 3 0 ]
  [ 1 3 0 2 ]
  [ 1 3 2 0 ]
  [ 2 0 1 3 ]
  [ 2 0 3 1 ]
  [ 2 1 0 3 ]
  [ 2 1 3 0 ]
  [ 2 3 0 1 ]
  [ 2 3 1 0 ]
  [ 3 0 1 2 ]
```

```
  [ 2 3 1 0 ]
  [ 3 0 1 2 ]
  [ 3 0 2 1 ]
  [ 3 1 0 2 ]
  [ 3 1 2 0 ]
  [ 3 2 0 1 ]
  [ 3 2 1 0 ]

------------------------------------------------
n=4, ONE edge (0->1): reduces permutations where 1 precedes 0
------------------------------------------------

Number of distinct topological sorts: 12
  [ 0 1 2 3 ]
  [ 0 1 3 2 ]
  [ 0 2 1 3 ]
  [ 0 2 3 1 ]
  [ 0 3 1 2 ]
  [ 0 3 2 1 ]
  [ 2 0 1 3 ]
  [ 2 0 3 1 ]
```

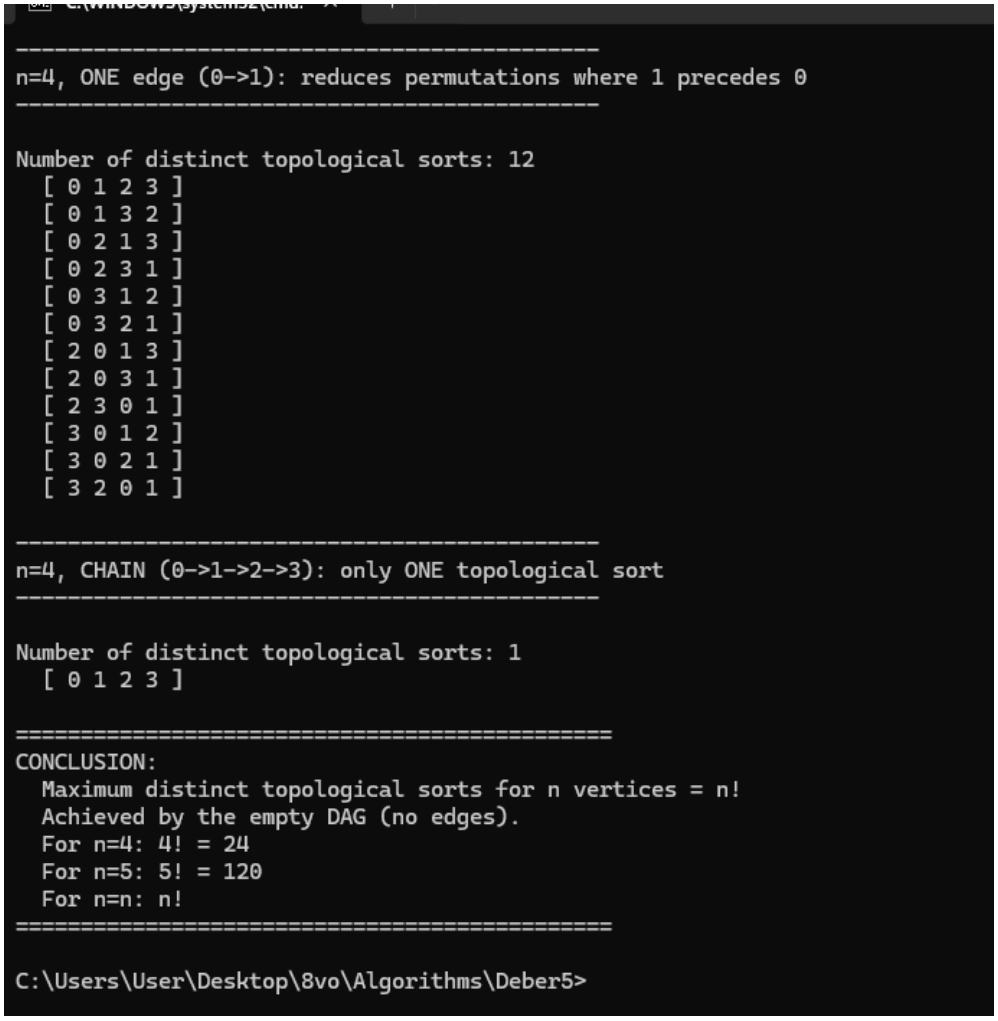


Figure 5: Execution output (Part 2): Topological sorts for a chain graph